

Project Name

Smart Fish Pond Management System.

Project Description

The Smart Fish Pond Management System is a connected monitoring and control solution built to help fish farmers keep their ponds healthy without needing to check on them constantly in person. A set of sensors placed in the pond continuously measures water temperature, acidity level (pH), water clarity, and rainfall. This information is sent wirelessly to a central server, where it is stored and made available in real time through a web based dashboard that farmers can open from a phone or computer.

Beyond monitoring, the system can also act on the pond's behalf. It automatically controls two important devices: an oxygen pump that keeps the water properly aerated, and a fish feeder that dispenses food on a schedule the farmer sets. Both can also be switched on or off manually whenever needed. When water conditions move outside a safe range, such as the temperature or pH drifting too far, the system sends an instant notification to the farmer's device so they can respond quickly, even if they are away from the pond.

The goal of the project is simple: reduce the risk of fish loss caused by poor water conditions going unnoticed, cut down the manual effort involved in daily pond care, and give fish farmers clear, timely information they can trust to make better decisions. The system is designed to run reliably over a local network, using a microcontroller connected to the sensors and devices, a backend service that processes and stores the data, and a lightweight web application that presents everything in an easy-to-understand way.

Technology Stack

Frontend

- Next.js 16
- Tailwind CSS
- Socket.IO Client (real time updates)

Backend

- Node.js with Express
- MongoDB with Mongoose

- Passport.js with Google OAuth 2.0

Microcontroller

- ESP32 Development Board (Wi-Fi enabled)

Sensors

- DS18B20 - water temperature sensor
- Analog pH sensor
- Analog turbidity sensor
- Analog raindrop sensor

Actuators

- Relay controlled oxygen pump (aeration)
- Relay controlled feeder motor
- Servo motor-feeder gate (opens and closes to release feed)

User Manual

Part 1: Prerequisites

Make sure you have:

- A computer or small server to run the backend (this stays on all the time, on your home or farm network). This server should have a public ip/domain so that the app is accessible from the internet.
- An ESP32 microcontroller with the sensors and actuators already wired up
- A Wi-Fi router that both the server and the ESP32 can connect to (the ESP32 only supports 2.4GHz Wi-Fi, not 5GHz)
- A phone, tablet, or computer to open the dashboard on.
- Clone the repository in <https://github.com/JUST-CSE-15-IoT-Group-2/smart-fish-pond-management-system> and remember the location.

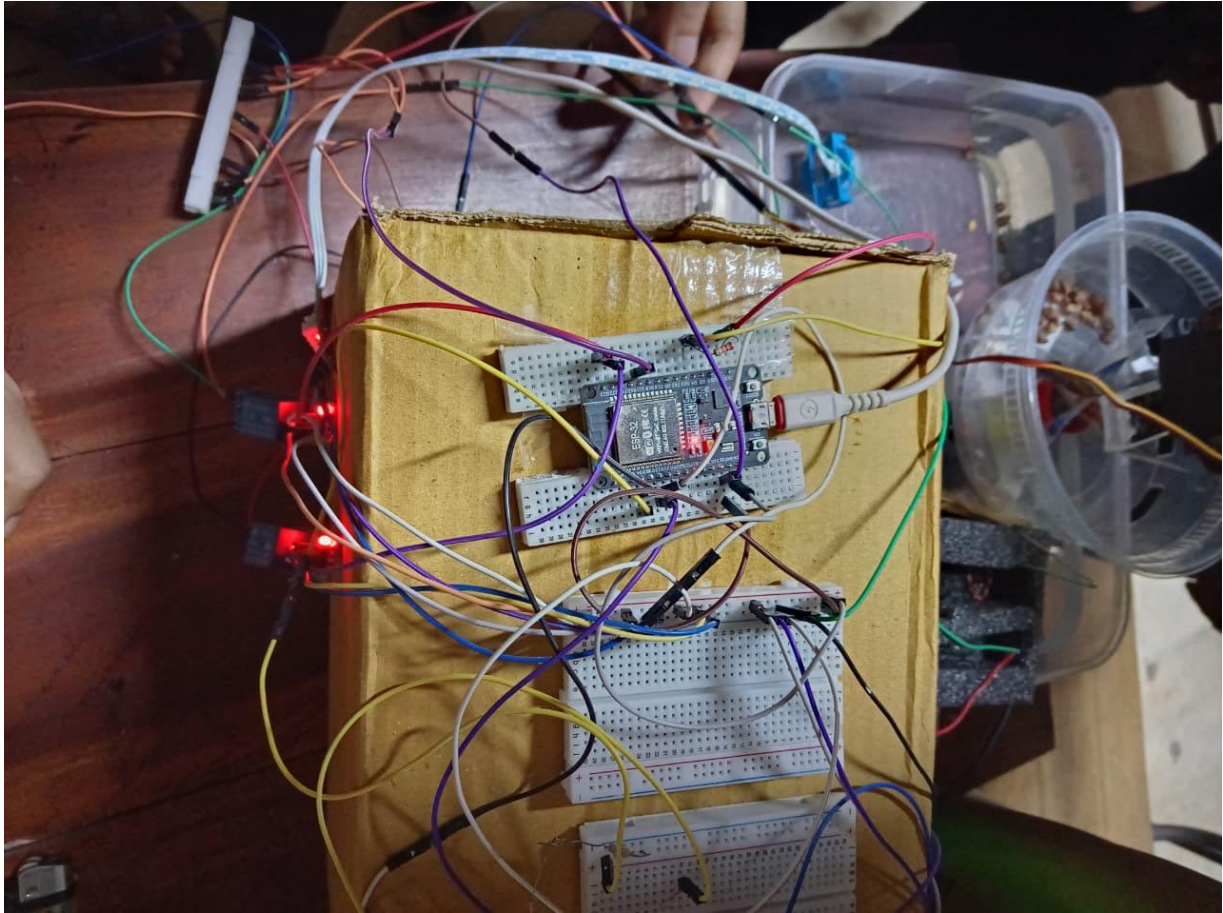


Figure 1: Hardware Setup.

Part 2: Setting Up the Backend

The backend is the program that stores your pond's data and talks to both the ESP32 and your dashboard.

1. Install Node.js and MongoDB on the computer that will act as your server.
2. Open the backend folder and create a `.env` file with your own values, including your database address and a secret key for login sessions. Also add the default username and password for login. The format is explained in `.env.example` file.
3. Open a terminal in backend folder and type **npm install**. This will install all required packages.
4. Start the backend by entering **npm run dev** command. Once running, it will print a message showing it is live and reachable on your network.
5. Make the backend server live on the internet and remember the address.

```
C:\WINDOWS\system32\cmd. X + v
PS C:\smart-fish-pond-management-system> cd backend
PS C:\smart-fish-pond-management-system\backend> npm install

up to date, audited 174 packages in 16s

27 packages are looking for funding
  run `npm fund` for details

7 vulnerabilities (1 low, 1 moderate, 5 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
PS C:\smart-fish-pond-management-system\backend> npm run dev

> fpms-backend@1.0.0 dev
> nodemon src/index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): src\**\*
[nodemon] watching extensions: js,json
[nodemon] starting `node src/index.js`
[MongoDB] Connected: localhost
[FPMS Backend] Running on http://0.0.0.0:5000 (all interfaces)
[FPMS Backend] LAN accessible - use your machine's local IP, e.g. http://192.168.x.x:5000
[FPMS Backend] WebSocket ready
[FPMS Backend] CORS: open to all origins
```

Figure 2: Backend Setup.

Keep this computer and the backend program running at all times. If it stops, your dashboard will lose its live connection and the ESP32 will not be able to send readings or receive commands.

Part 3: Setting Up the ESP32

The ESP32 is physically connected to your sensors and actuators.

1. Open the firmware file (from smart-fish-pond-management-system/esp32/esp_code folder) in the Arduino IDE.
2. Edit the Wi-Fi name and password in the code so the ESP32 can connect to your network.
3. Edit the address of your backend server (the one you found in Part 2) so the ESP32 knows where to send its readings.
4. Upload the code to the ESP32 using a USB cable.
5. Once it restarts, open the Serial Monitor in the Arduino IDE to watch it connect. You should see it join your Wi-Fi network and begin printing sensor readings every few seconds.

```
Output Serial Monitor X
Message (Enter to send message to 'ESP32 Dev Module' on 'COM3')
TEMP: 28.50 C
  POST temperature → HTTP -1
PH:  ADC=1474 V=1.188 pH=11.63 (ALKALINE)
  POST ph → HTTP -1
TURB: ADC=3121 V=2.515 NTU=23.8 [MODERATE]
  POST turbidity → HTTP -1
RAIN: 0.00% (DRY)
  POST rain → HTTP -1
----- Actuator Status -----
Oxygen Pump (GPIO 22) : OFF
Feeder Motor (GPIO 23) : OFF
Servo Gate (GPIO 21) : CLOSED
=====
TEMP: 28.62 C
```

Figure 3: ESP 32 Setup.

If the ESP32 cannot find your Wi-Fi, double check the network name is typed exactly right and that your router is broadcasting on the 2.4GHz band.

Part 4: Setting Up the Frontend Dashboard

The dashboard is the website you will check every day.

1. Start the frontend, or open the frontend folder.
2. Create a .env file with your own values. The format is explained in .env.example file.
3. Enter a terminal in the folder.
4. Enter command **npm install**.
5. Enter command **npm run dev**.
6. Log in using the id password you set on your backend .env file..

```
C:\WINDOWS\system32\cmd.exe x next-server (v16.2.9) x + v
PS C:\smart-fish-pond-management-system\frontend> npm install
up to date, audited 363 packages in 5s

143 packages are looking for funding
  run `npm fund` for details

9 high severity vulnerabilities

To address issues that do not require attention, run:
  npm audit fix

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
PS C:\smart-fish-pond-management-system\frontend> npm run dev

> frontend@0.1.0 dev
> next dev

▲ Next.js 16.2.9 (Turbopack)
- Local:      http://localhost:3000
- Network:    http://192.168.0.192:3000
- Environments: .env.local
✓ Ready in 5.6s
```

Figure 4: Frontend Setup.

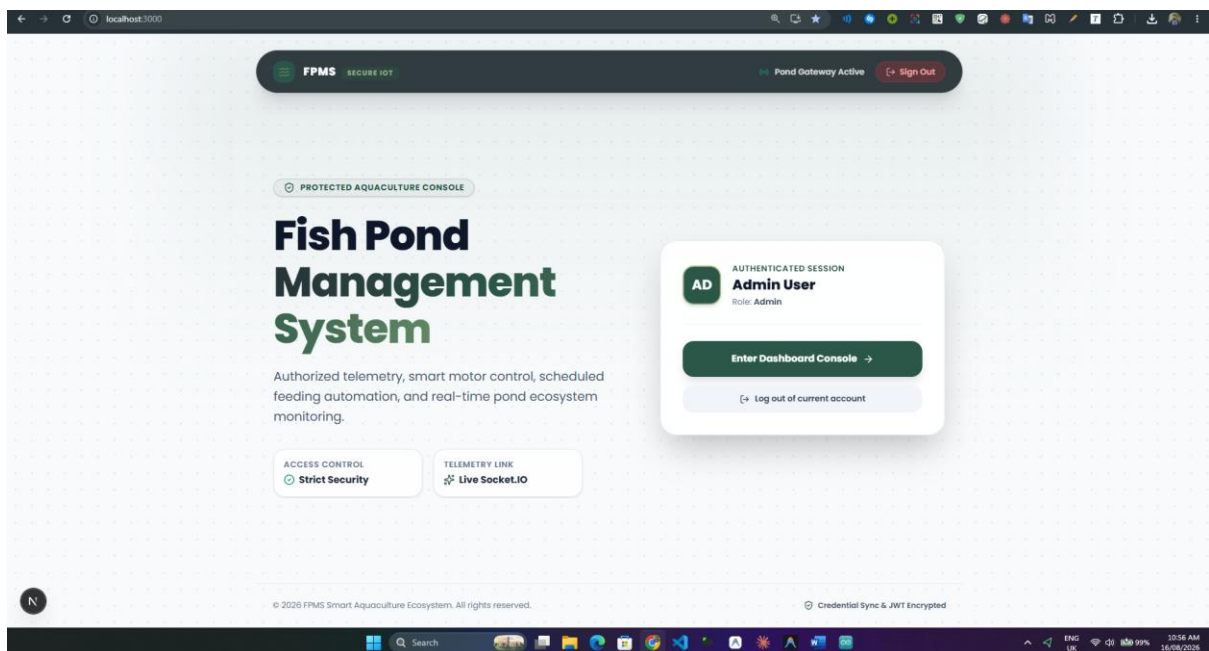


Figure 5: Landing Page.

Once logged in, you will land on your dashboard, ready to monitor and control your pond.

Part 5: Understanding the Dashboard

The dashboard has four main sections, reachable from the side menu.

Updates

This is your live view of the pond. You will see four cards showing water temperature, pH level, turbidity (clarity), and rainfall, all updating automatically as new readings arrive. Below the cards are trend charts so you can see how conditions have changed over recent readings. If a reading moves outside a safe range, the card will change color and you may also get a push notification on your device.

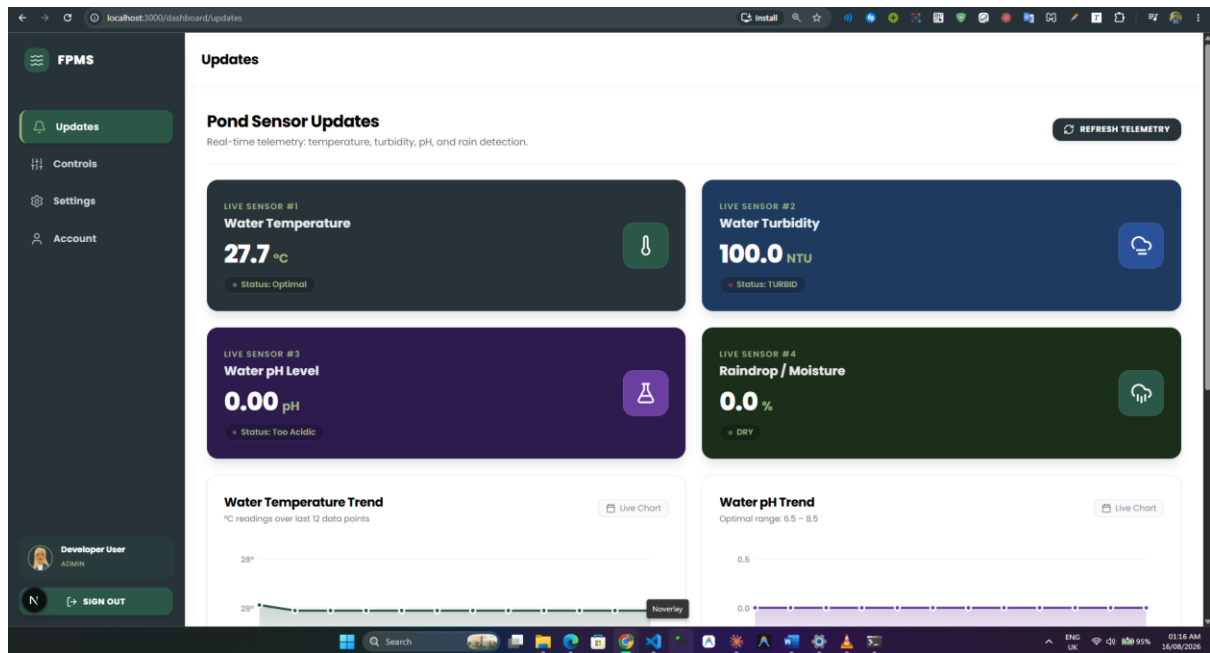


Figure 6: Updates menu from Dashboard.

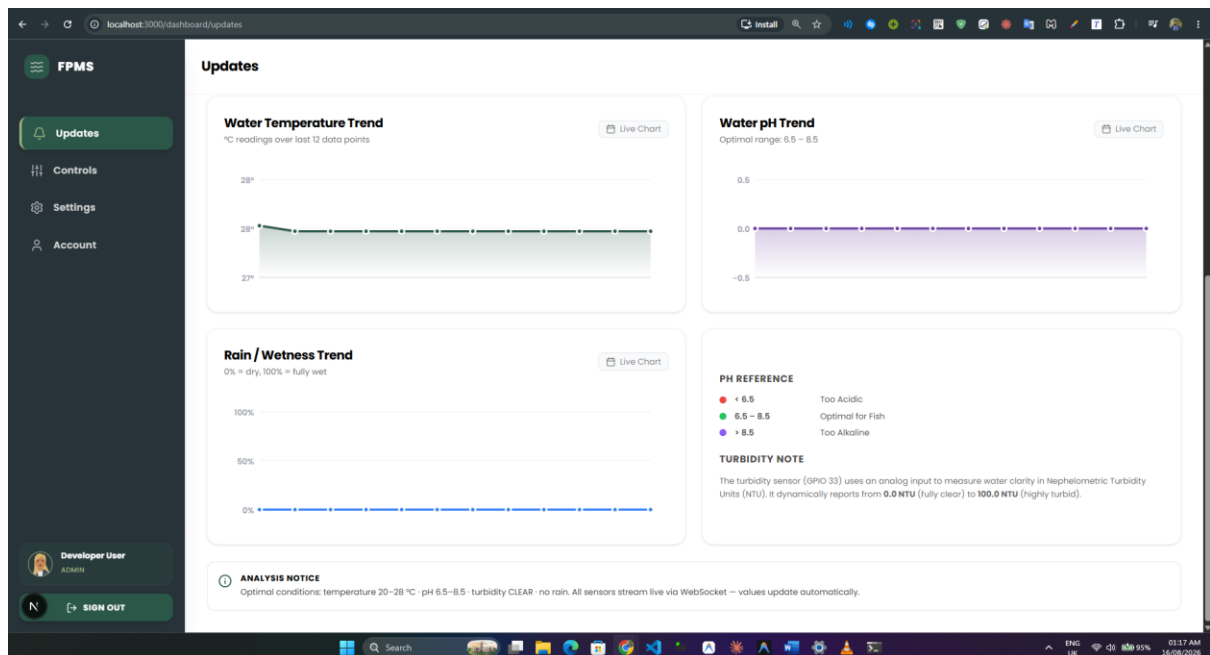


Figure 7: Updates menu from Dashboard - 2.

Controls

This is where you manage your feeder and oxygen pump.

- Feeding schedule: add or remove times of day when the feeder should run automatically, and set how many minutes it runs each time.
- Manual mode: turn on manual mode to take direct control of the feeder, then use the on and off button to feed your fish whenever you like, instead of waiting for the schedule.
- Oxygen pump: turn the pump on or off, and adjust its speed using the slider or the quick preset buttons. The pump will automatically turn on if rain meter detects more 40% rain and vice versa. User can adjust this from Settings.
- Connection link: if you ever need to stop all automatic actions immediately, you can sever the connection here. This safely shuts everything down until you reconnect it.

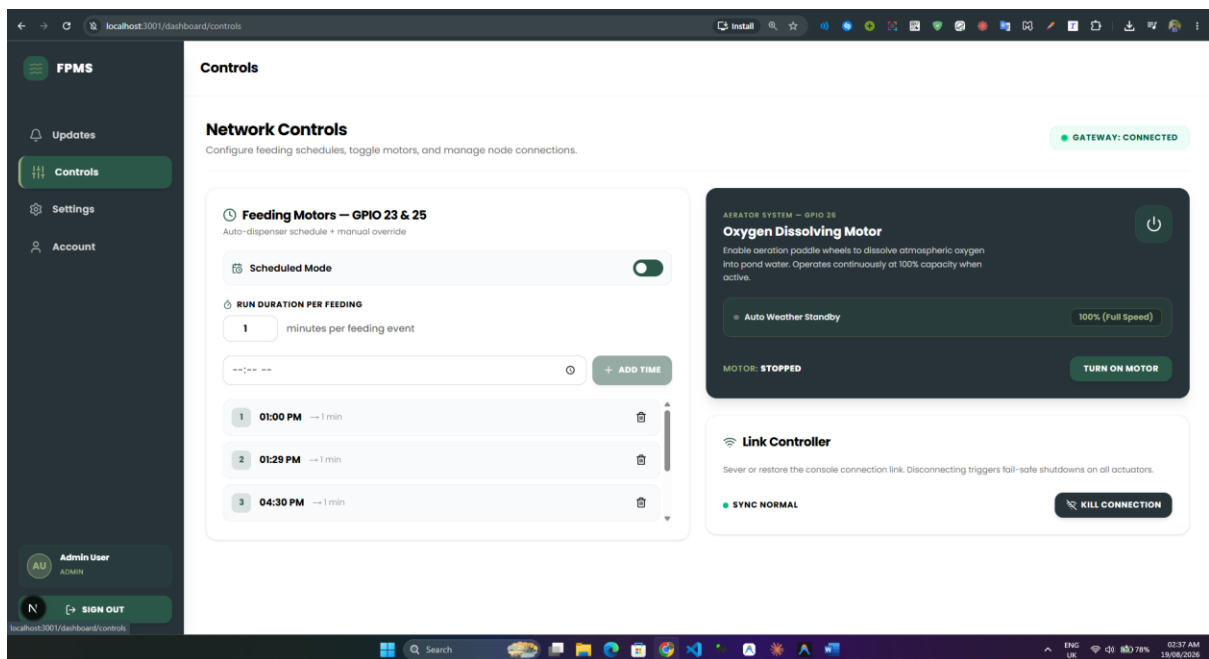


Figure 8: Controls menu from Dashboard.

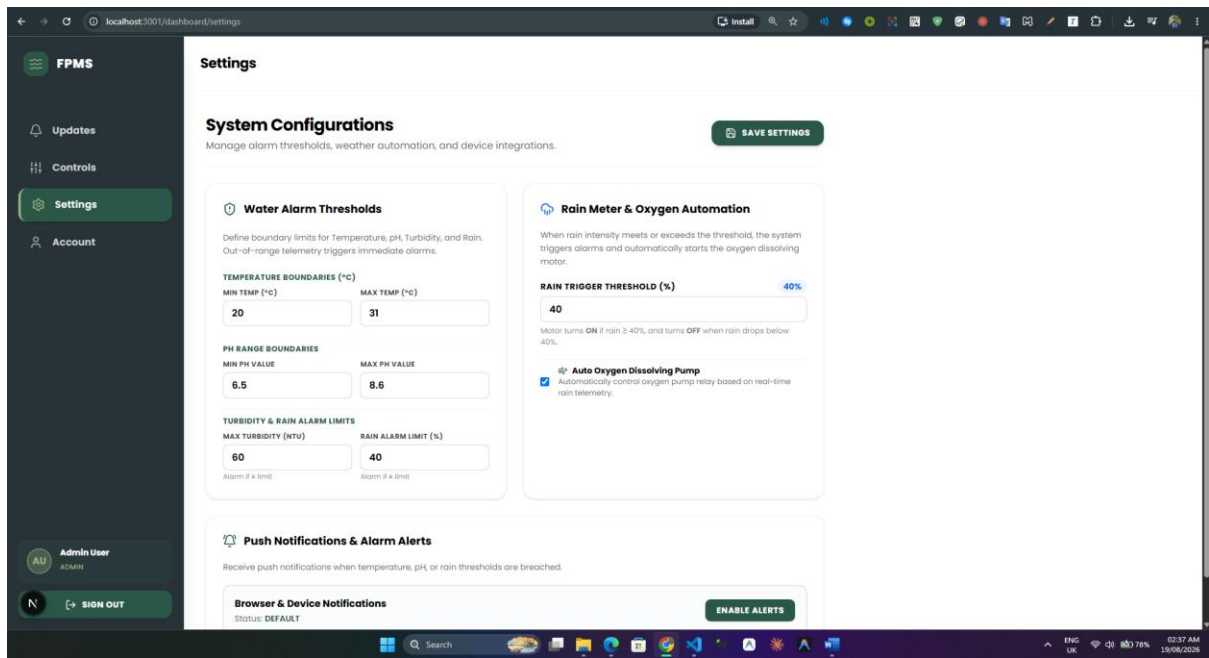


Figure 10: Settings Configuration.

Account

This shows your profile information and a personal access key.

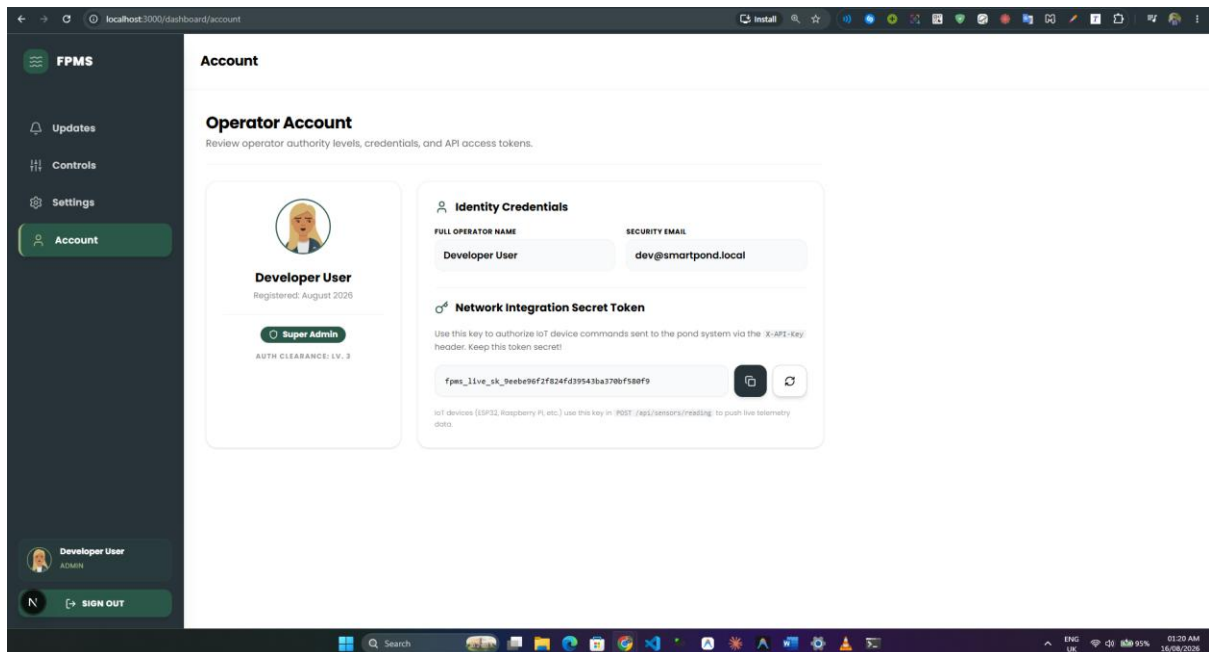


Figure 11: Account settings.

Part 6: Everyday Use

Once everything is set up, using the system day to day is simple.

- Check the Updates page whenever you want a quick look at your pond's condition.
- Let the feeding schedule run on its own, or switch to manual mode on days you want to feed by hand.
- Keep the oxygen pump running as needed, especially during hot weather or after rain.
- If you get a push notification about a warning, open the app and check the Updates page to see what triggered it.

Part 7: Simple Troubleshooting

No new readings showing up: Check that your server computer is turned on and the backend is running. Also check that the ESP32 is powered and connected to Wi-Fi.

Feeder or pump not responding: Make sure the connection link on the Controls page is not switched off. If it is, reconnect it.

Not receiving notifications: Make sure notifications are allowed for the dashboard in your phone or browser settings, and that you enabled alerts from the Settings page.

ESP32 will not connect to Wi-Fi: Confirm the Wi-Fi name and password in the firmware are correct, and that your router is on the 2.4GHz band, since the ESP32 cannot connect to 5GHz networks.

Conclusion

The project successfully built a working system that lets fish farmers monitor and control their pond remotely. It reads key water conditions, shows them live on a dashboard, and gives direct control over feeding and aeration, both scheduled and manual. Push notifications close the loop between detecting a problem and the user knowing about it. The biggest missing pieces are offline alerting and local schedule storage on the ESP32, and these should be the next priorities. Overall, the system proves that an affordable, locally deployable pond monitoring solution is achievable, and with the identified improvements, it has a clear path toward becoming a dependable tool for small and mid sized fish farms.